COURSE ANALYSIS · PREPARED 31 AUGUST 2026

AI Engineering Bootcamp

Complete syllabus, project catalogue and structural analysis

31	17	2	40+
modules documented	hands-on projects	distinct tracks	tools and frameworks

Scope note. This document reconstructs the syllabus from the course pages supplied. Modules 1 and 2 were not included in the source material and are therefore absent here; everything from Module 3 onward is complete. Project descriptions are taken from the dedicated projects page, which is more detailed than the in-module summaries and in several cases lists builds that do not appear against any module.

CONTENTS

Part One	**Track A — AI Engineering**	Modules 3 to 26
Part Two	**Track B — Deep Learning Foundations**	Modules 27 to 33
Part Three	**Hands-On Project Catalogue**	17 builds
Part Four	**Coverage Map & Technology Inventory**	2 tables
Part Five	**Structural Analysis**	9 findings

PART ONE

Track A — AI Engineering

Modules 3 to 26. Prompting, applications, retrieval, agents, production.

3 Prompt Engineering — Foundations to Advanced

Master the art and science of communicating effectively with LLMs.

- Structured outputs and custom response formats
- Conditional prompts and reusable prompt templates
- Zero-shot, one-shot, and few-shot prompting
- Chain-of-Thought, self-consistency, Auto-CoT, and persona-based prompting
- Prompt serialization formats such as Alpaca, ChatML, and INST
- Context engineering as the modern AI application design paradigm
- Context window management

4 Real-World Text Tasks & Chatbot Development

Apply LLMs to practical text-based use cases used in real products.

- Summarization, translation, tone control, and writing improvement
- Customer support ticket routing and analysis
- Code generation, modification, and explanation
- Role-playing chatbots, learning advisors, and support assistants

- **Project:** Working with the OpenAI API
- **Project:** 4A. Customer Support Agent — a production-style agent that understands user issues, classifies tickets, retrieves relevant help docs, escalates when required, and generates helpful responses

5 Structuring End-to-End LLM Applications

Learn how to build reliable and maintainable LLM-powered applications.

- JSON outputs and structured response handling
- Error handling and exception handling in AI applications
- Batching requests for efficiency
- Rate-limit retries and token-limit management

5A. FastAPI Basics for AI Applications

- FastAPI project setup
- Routes, request/response models, and Pydantic schemas
- Building AI API endpoints
- Error handling and environment configuration
- Connecting FastAPI with LLM workflows

6 Function Calling & Structured Outputs

Teach LLMs to interact with external systems using tools and typed outputs.

- Function calling and the tools parameter
- Multiple, parallel, and forced function calls
- Calling external APIs from inside the model loop
- Enforcing structured and typed outputs using Pydantic
- 6A. OpenAI SDK & Claude SDK for production AI apps — build real applications using both the OpenAI and Anthropic Claude SDKs

7 Embeddings & Semantic Search

Learn how AI applications understand meaning beyond keywords.

- How to create and use embeddings
- How to investigate and reason about vector spaces
- Semantic search and recommendation systems
- Embedding-based classification

■ **Project:** Exploring Embeddings

8 Vector Databases

Build storage and retrieval systems for AI-native applications.

- Vector database theory: construction, compression, and indexing
- ChromaDB end-to-end: CRUD, metadata filtering, and cost estimation
- Pinecone end-to-end: ingestion, querying, namespaces, and multitenancy
- Milvus DB overview

■ **Project:** RAG chatbot built on Pinecone and OpenAI over YouTube transcripts

9 Retrieval-Augmented Generation (RAG)

Build AI systems that can answer using private or domain-specific knowledge.

- RAG fundamentals: indexing workflow and retrieval mechanism
- Building RAG using LangChain loaders, chunking, and retrievers
- Hybrid search, reranking, and query rewriting
- RAGAS metrics and improving the "I don't know" rate

9A. LlamaIndex for Production RAG Applications

- LlamaIndex fundamentals
- Data connectors, documents, nodes, and indexes
- Query engines and retrievers
- Building RAG pipelines using LlamaIndex
- When to use LlamaIndex vs LangChain

■ **Project:** 9A. Enterprise Advanced RAG with hybrid search, re-ranking, and guardrails

10 Scalable RAG with Async Queues

Learn how to scale RAG systems beyond simple synchronous APIs.

- Sync vs async RAG architectures
- Python RQ with Redis/Valkey on Docker
- FastAPI chat queue design: enqueue, poll, and dequeue
- Worker orchestration and horizontal scaling

11 Hugging Face — Pre-Trained Models in Production

Use open-source AI models in practical production workflows.

- Hugging Face Hub, datasets, inference providers, and CLI
- Accessing instruction-tuned models such as Google Gemma
- AutoModels, AutoTokenizers, and AutoClasses
- Document Q&A pipelines with PyPDF

12 Multi-Modal AI with Hugging Face

Work with models that understand text, images, audio, and video.

- Computer vision tasks: image classification, object detection, and background removal
- Speech recognition, embeddings, denoising, and text-to-speech
- Zero-shot image and video classification using CLIP and CLAP
- Visual Question Answering, Document VQA, diffusion, inpainting, and video models

13 AI Agents — From First Agent to Multi-Agent Systems

Build intelligent systems that can reason, use tools, and complete multi-step tasks.

- Agentic AI fundamentals
- Building your first AI agent
- Hugging Face smolagents
- Custom tools using @tools
- Web search and data access tools
- Agentic RAG with stateful RAG search tools
- Multi-agent systems with specialists, coordination, and memory
- Workflow patterns: chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer
- 13A. AutoGen & CrewAI for multi-agent applications — popular frameworks for collaborative AI systems with specialized agents
- 13B. A2A Protocol & spinning up subagents — A2A and subagent orchestration as a dedicated advanced section

14 LangGraph & Stateful Agent Workflows

Design advanced agent workflows with state, memory, and persistence.

- LangGraph core concepts: nodes, edges, state, and StateGraph
- Designing complex agent graphs with langchain and LCEL
- Debugging stateful agent workflows
- Checkpointing workflows with MongoDB for persistence

14A. LangFlow for Visual AI Workflow Building

- Building AI workflows visually
- Connecting LLMs, prompts, tools, retrievers, and memory
- Rapid prototyping of RAG and agent workflows
- Exporting and understanding generated workflow logic
- When LangFlow is useful in real teams

- **Project:** 14A. Claude Code-like Coding Agent — understands a codebase, inspects files, makes edits, runs commands, and iterates on fixes

15 The Memory Layer in AI Agents

Give AI agents the ability to remember, retrieve, and reason across interactions.

- Short-term, long-term, factual, episodic, and semantic memory
- Mem0 setup and configuration
- Vector-database-backed memory for agents
- Deep agents and deep research agents

16 Graph Memory & Knowledge Graphs

Use graph databases to build richer and more connected AI memory systems.

- Why graph memory matters for agents
- Neo4j and Kuzu for graph storage
- Cypher query basics
- Adding and testing graph memory in an AI agent

17 Multi-Modal Agents & Voice Agents

Build agents that can see, hear, speak, and respond across modalities.

- Multi-modal agents and image-based reasoning
- Speech-to-Speech voice agents
- Chained voice agents: STT to LLM to TTS

18 Model Context Protocol (MCP) & Agent SDK

Understand the modern tooling ecosystem around AI agents.

- What MCP is and why it matters
- MCP architecture
- Building a tiny MCP server
- Agent SDK concepts: Hosted Tools, Function Tools, and Agent as a Tool

- **Project:** 18A. AI Code Reviewer Agent like CodeRabbit — analyzes pull requests, understands diffs, detects bugs, suggests improvements, and posts structured review comments

19 Streaming, Retries, Fallbacks & Rate Limits

Build robust AI APIs that work reliably in production.

- Streaming responses using Server-Sent Events (SSE)
- Exponential backoff and retry strategies
- Idempotency keys
- Token budgets
- Building a multi-provider LLM Gateway with automatic fallback

20 Local LLM Deployment

Run and deploy models locally using modern open-source tooling.

- Ollama as a local LLM runtime engine
- Dockerized LLM environments
- OpenWebUI
- Building FastAPI services on top of Ollama

21 Prompt Caching & Cost / Latency Optimization

Optimize AI applications for speed, cost, and scale.

- Anthropic and OpenAI prompt-caching mechanics
- Prefix-based caching and cache-friendly prompt design
- KV cache fundamentals
- Batching, prefill, and decode phases

22 Observability & Tracing

Monitor, debug, and improve AI applications like production systems.

- Why traditional logs are not enough for agentic apps
- Langfuse, LangSmith, Braintrust, and OpenLLMetry
- OpenTelemetry GenAI semantic conventions
- Tracing model calls, tools, retrieval, and agent workflows

23 Evaluations — The Underrated AI Engineering Skill

Learn how to systematically measure and improve AI application quality.

- Trace-by-hand workflow on production traces
- Failure bucketing
- Binary pass/fail evaluations
- LLM-as-judge prompts
- Achieving strong judge alignment
- The three gulfs model: comprehension, specification, and generalization
- RAGAS metrics in practice

24 Security, Guardrails & Responsible Deployment

Build AI products that are safer, more reliable, and production-ready.

- OWASP Top 10 for LLM Applications
- Defense-in-depth for AI systems
- Least-privilege tool design
- Schema validation
- Human-in-the-loop workflows
- NeMo Guardrails, Llama Guard, and ShieldGemma
- Prompt injection testing
- Hands-on: prompt-injecting your own agent

25 MLOps & LLMOps in Production

Understand the complete lifecycle of deploying and maintaining AI systems.

- MLOps lifecycle, roles, deployment, monitoring, and drift
- LLMOps lifecycle: ideation, development, and operations
- Cost management and prompt compression
- Governance and production readiness
- Choosing between RAG and fine-tuning

26 Production Best Practices

Learn the final layer of practices required to ship reliable AI products.

- Moderation APIs
- Prompt-injection mitigation
- Guardrails and output validation
- Adversarial testing
- Minimizing model risks in production

- **Project:** AI Trading Agent — analyzes market data, reads financial news, reasons over signals, and generates structured trade ideas with risk controls (listed elsewhere as a bonus homework project)

PART TWO

Track B — Deep Learning Foundations

Modules 27 to 33. Mathematics, PyTorch, vision, sequences, transformers, fine-tuning.

27 Math & PyTorch Foundations

Build the mathematical and programming foundation required for deep learning.

- Linear algebra, probability, statistics, and calculus
- Gradient descent
- PyTorch tensors, layers, weights, and parameters
- Training loops, optimizers, learning rate, and momentum
- Overfitting, dropout, and model evaluation with torchmetrics

- **Project:** Simple Neural Network in TensorFlow (mini project)
- **Project:** MNIST Autoencoder (mini project)

28 Robust Deep Learning & Computer Vision

Learn how to structure deep learning projects cleanly and train vision models.

- PyTorch with object-oriented programming
- Clean Dataset, DataLoader, Model, and Trainer classes
- Vanishing and exploding gradients
- ReLU vs ELU
- Batch Normalization
- CNNs end-to-end
- Convolutional layers, augmentation, training, and evaluation

29 Sequence Modelling & Multi-IO Architectures

Understand models that work with sequences and multiple inputs or outputs.

- RNNs, LSTMs, and GRUs
- Forecasting with RNNs
- Multi-input and multi-output architectures
- Loss weighting for complex models

- **Project:** Developing Multi-Input Models for OCR

30 Transformers from Scratch

Understand the architecture that powers modern LLMs.

- Self-attention
- Positional encoding
- Multi-head attention
- Masked attention
- Full Transformer architecture
- Encoder-only, decoder-only, and encoder-decoder models
- Pretraining objectives: MLM and CLM
- GPT vs BERT
- Building a custom BPE tokenizer

31 Deep Generative Models

Learn how modern generative AI models are built.

- GAN fundamentals
- Implementing a GAN for handwritten digit generation
- Variational Autoencoders (VAEs)
- Training a face-generation VAE on CelebA
- TensorBoard for generative-model metrics

32 LLM Internals — Theory to Match the Practice

Go deeper into the engineering and architecture choices behind modern LLMs.

- KV Cache
- Paged Attention
- Flash Attention
- Sparse Attention
- Quantization
- Mixture of Experts
- Distillation
- Speculative Decoding
- RLHF
- Chain of Thought and Tree of Thought
- DeepSeek architecture case study

33 Fine-Tuning, LoRA / QLoRA & Trending Topics

Learn how to adapt existing models for domain-specific tasks.

- Pretraining vs fine-tuning
- When to use fine-tuning and when not to
- LoRA and QLoRA
- Parameter-efficient fine-tuning
- Hands-on fine-tuning with Unsloth
- Diffusion models
- Vision Transformers
- CLIP recap

PART THREE

Hands-On Project Catalogue

All seventeen builds, with the module each belongs to and the stack each demands.

01 Working with the OpenAI API

CORE · MODULE 4

Build production-style Python utilities on top of the OpenAI API for summarization, translation, tone control, ticket routing, and code generation.

`Stack: OpenAI API, Python`

02 Customer Support Agent

CORE · MODULE 4A

Build a production-style customer support agent that classifies tickets, retrieves help docs via RAG, drafts replies, and escalates to a human when the model is out of its depth.

`Stack: OpenAI API, RAG, vector store, routing logic`

03 Exploring Embeddings

CORE · MODULE 7

Build semantic search, recommendation, and classification systems powered by embeddings, and learn to reason about the vector space behind them.

`Stack: Embedding models, numpy, visualization`

04 RAG Chatbot over YouTube Transcripts

CORE · MODULE 8

Build a Pinecone-backed RAG chatbot that ingests YouTube transcripts, retrieves the relevant timestamps, and answers questions grounded in what was actually said.

`Stack: Pinecone, OpenAI, transcript ingestion, chunking`

05 Enterprise Advanced RAG

FLAGSHIP · MODULE 9A

Build an enterprise-grade RAG stack with hybrid search (BM25 + vector), cross-encoder reranking, guardrails, and end-to-end RAGAS evaluation on real corporate data.

`Stack: LangChain, BM25, cross-encoder reranker, RAGAS, guardrails`

06 Claude Code-like Coding Agent

FLAGSHIP · MODULE 14A

Build a Claude Code-style coding agent that understands a codebase, inspects files, makes edits, runs commands in a sandbox, and iterates on fixes for hours at a time.

`Stack: LangGraph, MongoDB checkpointing, sandboxed execution, tool calling`

07 AI Code Reviewer Agent like CodeRabbit

FLAGSHIP · MODULE 18A

Build a CodeRabbit-style AI reviewer that reads every pull request, flags bugs and security issues with RAG over your style guide, and posts structured inline comments via the GitHub API.

`Stack: MCP / Agent SDK, GitHub API, RAG, structured outputs`

08 Voice-Based AI Agent

FLAGSHIP · MAPS TO MODULE 17

Build a chained voice agent (STT to LLM to TTS) that holds natural conversations, calls tools, and runs as a Cursor-style voice coding assistant.

`Stack: STT, TTS, tool calling, streaming`

09 AI Trading Agent

BONUS HOMEWORK · MODULE 26

Build a trading research agent that ingests market data and financial news, reasons over signals via agentic RAG, and proposes structured trade ideas with risk controls for humans to act on.

`Stack: Agentic RAG, market data APIs, structured outputs, risk rules`

10 Simple Neural Network in TensorFlow

MINI · MODULE 27

Build a feedforward neural network in TensorFlow from scratch: your first end-to-end model with forward pass, training loop, and evaluation.

`Stack: TensorFlow`

11 MNIST Autoencoder

MINI · MODULE 27

Train an autoencoder on the MNIST dataset to learn unsupervised representation learning and reconstruction: your first generative model.

`Stack: PyTorch / TensorFlow, MNIST`

12 Developing Multi-Input Models for OCR

CORE · MODULE 29

Build a multi-input PyTorch model that fuses image and metadata streams to perform end-to-end OCR, and learn how to weight losses across multiple heads.

`Stack: PyTorch, CNN + sequence heads, multi-loss training`

13 ChatGPT-Style Chatbot Platform

FLAGSHIP · UNMAPPED

Build a full ChatGPT-style chatbot with a Streamlit UI, persistent multi-turn memory, web search tools, and a ChromaDB vector store as long-term knowledge.

`Stack: Streamlit, ChromaDB, memory layer, web search tools`

14 Chat With Any Document

CORE · UNMAPPED

Build a ChatPDF-style app that turns research papers, contracts, and textbooks into a conversation grounded entirely in their source.

`Stack: PDF parsing, chunking, vector store, RAG`

15 AI Search Engine like Perplexity

FLAGSHIP · UNMAPPED

Build a conversational search engine with agentic web retrieval, hybrid search, inline citations, streaming answers, and faithfulness evaluation.

`Stack: Agentic retrieval, hybrid search, SSE streaming, eval harness`

16 AI Web App Builder like Lovable / v0

FLAGSHIP · UNMAPPED

Build a prompt-to-app system where a user describes a web app and a multi-agent pipeline (planner, coder, reviewer) ships a working version with live preview.

`Stack: Multi-agent orchestration, code generation, sandboxed preview`

17 AI Interview Taker like Mercor

FLAGSHIP · UNMAPPED

Build an AI that conducts real technical and behavioral interviews over voice, scores candidates against a rubric with Pydantic, and generates a structured post-interview hiring report.

`Stack: Voice pipeline, Pydantic rubric scoring, report generation`

PART FOUR

Coverage Map & Technology Inventory

Which project belongs to which module, and every tool the course touches.

Project to module mapping

Project	Module	Type
Working with the OpenAI API	Module 4	Core
Customer Support Agent	Module 4A	Core
Exploring Embeddings	Module 7	Core
RAG Chatbot over YouTube Transcripts	Module 8	Core
Enterprise Advanced RAG	Module 9A	Flagship
Claude Code-like Coding Agent	Module 14A	Flagship
AI Code Reviewer Agent like CodeRabbit	Module 18A	Flagship
Voice-Based AI Agent	Maps to Module 17	Flagship
AI Trading Agent	Module 26	Bonus homework
Simple Neural Network in TensorFlow	Module 27	Mini
MNIST Autoencoder	Module 27	Mini
Developing Multi-Input Models for OCR	Module 29	Core
ChatGPT-Style Chatbot Platform	Unmapped	Flagship
Chat With Any Document	Unmapped	Core
AI Search Engine like Perplexity	Unmapped	Flagship
AI Web App Builder like Lovable / v0	Unmapped	Flagship
AI Interview Taker like Mercor	Unmapped	Flagship

Projects shown in orange do not appear against any module in the supplied syllabus.

Technology stack inventory

Category	Tools and techniques
Languages & core	Python, SQL-adjacent query languages (Cypher)
Backend & serving	FastAPI, Pydantic, Server-Sent Events, Docker, Python RQ, Redis / Valkey
Model providers & SDKs	OpenAI API and SDK, Anthropic Claude SDK, Hugging Face Hub and inference providers, Google Gemma, Ollama
Agent frameworks	LangChain, LCEL, LangGraph, LangFlow, LlamaIndex, smolagents, AutoGen, CrewAI, Agent SDK
Protocols	Model Context Protocol (MCP), Agent-to-Agent (A2A)
Vector & graph stores	ChromaDB, Pinecone, Milvus, Neo4j, Kuzu, MongoDB (checkpointing)

Category	Tools and techniques
Memory	Mem0, vector-DB-backed memory, graph memory
Retrieval techniques	Chunking, hybrid search (BM25 + vector), cross-encoder reranking, query rewriting
Observability	Langfuse, LangSmith, Braintrust, OpenLLMetry, OpenTelemetry GenAI conventions
Evaluation	RAGAS, LLM-as-judge, binary pass/fail evals, failure bucketing, three gulfs model
Safety & guardrails	OWASP Top 10 for LLMs, NeMo Guardrails, Llama Guard, ShieldGemma, moderation APIs
Deep learning	PyTorch, TensorFlow, torchmetrics, TensorBoard, Unsloth, CLIP, CLAP
UI	Streamlit (the only interface layer named anywhere in the course)
External APIs	GitHub API, web search tools, market data and financial news feeds

PART FIVE

Structural Analysis

What the syllabus does well, what it omits, and where it is internally inconsistent.

01 The course is applications-first and theory-last

Modules 3 to 26 form a complete AI engineering track: prompting, then applications, then retrieval, then agents, then production. Modules 27 to 33 are a separate deep learning track that starts again from linear algebra. The two are stacked, not interleaved. You ship working systems long before you learn what is inside the model, which is the right order for someone arriving from software engineering, but it means the last seven modules read like a second, different course.

02 Seventeen projects, but only twelve map to a module

Five of the strongest builds — the ChatGPT-style platform, Chat With Any Document, the Perplexity-style search engine, the Lovable/v0-style app builder, and the Mercor-style interview taker — do not appear in any module in the syllabus. They are capstones assembled from techniques taught across several modules. Worth confirming with the bootcamp whether these are taught live, assigned as homework, or left as self-directed builds.

03 The portfolio framing is deliberate

Six projects are explicitly clones of named products: Claude Code, CodeRabbit, Perplexity, Lovable/v0, Mercor, and ChatPDF. That is strong signalling on a resume because a reviewer instantly understands the scope. It also means the bar is high; a half-finished Perplexity clone reads worse than a polished smaller build.

04 Streamlit is the only UI in the entire course

Across 31 modules and 17 projects, the only interface layer mentioned is Streamlit, in one project. Everything else stops at the FastAPI boundary. There is no streaming chat UI, no token-by-token rendering, no React or Next.js integration, no design work on agent interfaces. For a frontend engineer this is the single largest arbitrage in the syllabus: the course gives you the backend half you lack and leaves untouched the half you already own.

05 Deployment and infrastructure are thin

Docker appears in modules 10 and 20, and module 25 covers the MLOps lifecycle conceptually. There is no cloud provider, no CI/CD, no Kubernetes, no infrastructure-as-code. "Production" in this syllabus means robust application code — retries, guardrails, tracing, evals — not running infrastructure. That is a defensible definition, but do not expect to come out able to deploy and operate a service end to end.

06 Agent frameworks are covered broadly rather than deeply

smolagents, LangGraph, LangFlow, AutoGen, and CrewAI all appear inside modules 13 and 14. That is a great deal of surface area for two modules. Expect an overview of each and real depth in one or two. LangGraph is the one worth going deep on, since the Claude Code-style coding agent is built on it.

07 Several project technologies are never taught

The projects assume BM25, cross-encoder reranking, sandboxed command execution, the GitHub API, Streamlit, and market data ingestion. None of these appear as a module topic. Module 9 says "hybrid search" and "reranking" without naming the implementations. Budget self-study time for the gap between what a module teaches and what its project actually requires.

08 Two numbering collisions and one framework inconsistency

"9A" labels both the Enterprise Advanced RAG project and the LlamaIndex section. "14A" labels both the Claude Code-like coding agent project and the LangFlow section. Separately, module 27 is titled "Math & PyTorch Foundations" but its first mini project is built in TensorFlow, which suggests material carried over from an earlier version of the course. None of this affects content quality; it will affect your ability to track progress against a checklist.

09 Evaluation and observability are the sleeper modules

Modules 22 and 23 — tracing and evals — are where most self-taught AI engineers are weakest, because they are unglamorous and there is no tutorial dopamine in them. They are also what separates someone who can demo an agent from someone who can be trusted to own one in production. If you skim anything, do not skim these.

Where this leaves you

The syllabus is genuinely current — MCP, A2A, LangGraph, Mem0, graph memory, OpenTelemetry GenAI conventions and the three gulfs eval model are all recent additions to the field, not filler. The production modules on tracing, evaluation and guardrails are the ones that separate this from a tutorial playlist. The deep learning track at the end is thorough but optional for an application engineer; it matters if you want to reason about model behaviour rather than only orchestrate models.

The one thing to hold onto: nothing in 31 modules teaches the interface layer. Every project stops at an API or a Streamlit page. If you pair this backend curriculum with the frontend work you already do — streaming chat surfaces, tool-call rendering, agent state made legible to a user — you end up in a much rarer position than someone who completes the course alone.